sust-AI-nable

Multi-Agent Cloud Carbon Optimization System

Problem: Cloud workloads and AI pipelines generate significant carbon emissions, yet developers and organizations lack tools that translate emissions data into actionable infrastructure decisions.

Our Goal: Build an AI-agent system that retrieves carbon data, analyzes workloads, and recommends lower-carbon execution strategies with explainable reasoning.

What We Needed to Validate

Key Assumptions from CP1

- ✓ Developers would benefit from carbon-aware infrastructure recommendations
- ✓ Sustainability dashboards are too complex for decision making
- ✓ AI agents could assist in reasoning about sustainability trade-offs

Validation Methods

- ✓ Pilot interviews with developers and governance stakeholders
- ✓ Prompt experiments using LLM tools
- ✓ Prototype dashboard simulation

Prompting Protocol & Validation Design

AI Tools Tested

- ✓ Cursor – system architecture & simulation data
- ✓ GitHub Copilot – debugging multi-agent pipeline
- ✓ NotebookLM – research synthesis
- ✓ ChatGPT – sustainability reasoning

Prompt Categories

Typical Case: Carbon-aware workload scheduling

Edge Case: Cost vs carbon trade-off reasoning

Failure Case: 30-day multi-agent simulation

An organization runs AI workloads across three cloud regions.

Region A carbon intensity: 450 gCO2/kWh
Region B carbon intensity: 180 gCO2/kWh
Region C carbon intensity: 320 gCO2/kWh

A training job consumes approximately 200 kWh of electricity.

Which region should the organization choose to minimize carbon emissions while maintaining similar performance? Explain the reasoning behind your choice.

Choose **Region B**.

Here's the carbon impact for the same 200 kWh training job in each region:

- **Region A:** 450 gCO2/kWh × 200 kWh = **90,000 gCO2 = 90 kgCO2**
- **Region B:** 180 gCO2/kWh × 200 kWh = **36,000 gCO2 = 36 kgCO2**
- **Region C:** 320 gCO2/kWh × 200 kWh = **64,000 gCO2 = 64 kgCO2**

Why Region B?
Because it has the **lowest carbon intensity**, so the same workload produces the **least emissions** there.

Compared with the others, choosing Region B would avoid:

- **54 kgCO2** versus Region A
- **28 kgCO2** versus Region C

So if performance is similar across regions, the best choice is **Region B**, since it minimizes emissions without sacrificing workload quality.

Typical Case

A company can run a workload in two cloud regions.

Region X
Carbon intensity: 210 gCO2/kWh
Cost: $0.90 per compute hour

Region Y
Carbon intensity: 180 gCO2/kWh
Cost: $1.05 per compute hour

The job requires 8 compute hours.

Which region should the company select if they want to balance cost and carbon emissions? Explain how the decision should be made.

First compute **both the cost and carbon impact** of running the 8-hour job in each region.

Assume the workload consumes the same energy per hour in both regions (performance is similar), so only **carbon intensity and price** differ.

1. Cost Comparison

Region X

$$8 \text{ hours} \times \$0.90 = \$7.20$$

Region Y

$$8 \text{ hours} \times \$1.05 = \$8.40$$

Cost difference:

$$\$8.40 - \$7.20 = \$1.20$$

4. Practical Recommendation

Because:

- Region Y emits **~14% less carbon**
- The cost increase is **small ($1.20 total)**

A company prioritizing **sustainability with minimal financial impact** should choose **Region Y**.

However:

- If cost sensitivity is very high → choose **Region X**
- If ESG or internal carbon pricing matters → choose **Region Y**

✅ **Balanced choice: Region Y**, because it meaningfully reduces emissions with only a small cost increase.

Edge Case

Validation Reflection

Team Member	Contribution	Key Insights	Design Impact
Pooja	Professional interviews with cloud engineers, governance professionals, and AI developers	• Sustainability is rarely considered during system design because existing tools are retrospective and difficult to interpret. • Engineers lack visibility into energy impact and developers receive no feedback on carbon impact.	**Led to the idea of real-time sustainability decision support integrated into engineering workflows.**
Sarthak	Industry interviews with data engineers and risk professionals	• Carbon data is fragmented across infrastructure layers and difficult to retrieve. • Existing tools lack developer-focused sustainability guidance and natural language interfaces.	**Motivated the use of a multi-agent system capable of retrieving emissions data and generating recommendations.**
Josue	User interviews on carbon awareness and footprint tools	• Users care about environmental impact but rarely calculate their carbon footprint because tools require too much data and lack transparency. • Participants preferred simple inputs and actionable recommendations rather than just emissions numbers.	**Reinforced the need for simple interaction and explainable recommendations rather than complex dashboards.**

Evidence

What Worked?

- ✓ Identified lower-carbon cloud regions
- ✓ Generated sustainability optimization suggestions
- ✓ Explained carbon-aware scheduling decisions
- ✓ Helped prototype system architecture

What Failed?

- ✓ Token limits during multi-agent simulations
- ✓ API rate limits during long orchestration cycles
- ✓ Agents repeated reasoning loops without new data
- ✓ Emission factor sources were rarely cited

Gap Analysis & Opportunity

Platform	What They Do Well	Key Limitation	Opportunity for Our System
Plan A	Carbon accounting, ESG reporting, Scope 1–3 emissions tracking	Focuses on reporting and compliance dashboards	Lack of real-time infrastructure optimization
Normative	Accurate carbon footprint calculation and regulatory reporting	Primarily used for enterprise reporting and sustainability teams	Does not guide developers on compute-level decisions
Traditional Carbon Tools	Measure and report emissions data	Post-hoc analysis after emissions occur	No automated decision support

Most platforms measure carbon emissions.
Few systems actively guide how to reduce them in real-time.

Concept Evolution

Initial Idea: Multimodal AI scheduling assistant

The system was only predicting recommendations, not making decisions.

Refined Concept: Autonomous sustainability decision agent

Agents decide:

- **Where** workloads should run
- **When** they should run
- **How** to balance carbon, cost, and performance

team-project-carbon-emissions

EXPLORER

OPEN EDITORS
- SYSTEM_DESIGN.md

TEAM-PROJECT-CARBON-EMIS...
- __pycache__
- data
- literature
  - full literature
- proposal
  - SustAInaibility.pptx
- reflections
  - full reflections
- src
- tests
- validation
- .env.example
- .gitignore
- CODE_WALKTHROUGH.md
- config.py
- dashboard.py
- Design_SPEC.md
- README.md
- requirements.txt
- ROADMAP.md
- run_baseline.py
- run_pipeline.py
- SOURCES.md
- SYSTEM_DESIGN.md

OUTLINE

TIMELINE

SYSTEM_DESIGN.md

SYSTEM_DESIGN.md > # sust-AI-naible: Multi-Agent Cloud Carbon Optimization System > ## 4. Optimization Framing > ### Gamification ↔ Verified Savings Connection

```
# sust-AI-naible: Multi-Agent Cloud Carbon Optimization System
## 1. Agent Roster
### Architecture Overview
```

```
│                                                    │
│  ┌──────────────────────────────────────────┐      │
│  │            SHARED MEMORY (World Model)   │      │
│  │  Activity Ledger │ Decision Log │ Outcome Log │  │
│  └──────────────────────────────────────────┘      │
│                          │                         │
│                          ▼                         │
│                    ┌──────────┐                    │
│                    │ DEV COPILOT │                  │
│                    │ (optional)  │                  │
│                    └──────────┘                    │
└────────────────────────────────────────────────────┘
```

Agent 1: Ingestor Agent

- **Purpose**: Collect, normalize, and validate cloud usage data from heterogeneous sources into a canonical activity ledger.
- **Type**: Mostly deterministic, LLM-assisted for schema mapping of new sources.

Aspect	Detail
Inputs	Cloud billing APIs (AWS CUR, GCP Billing Export, Azure Cost Mgmt), K8s metrics (Prometheus), CI/CD logs (GitHub Actions, Jenkins), scheduler metadata
Outputs	Normalized activity records: `{job_id, service, region, timestamp_start, timestamp_end, resource_type, resource_qty, cost_usd, workload_category, team_id}`

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
  Verified 3,500 / 5,478 recommendations...
  Verified 4,000 / 5,478 recommendations...
  Verified 4,500 / 5,478 recommendations...
  Verified 5,000 / 5,478 recommendations...
  Verification complete: 5478 verified (1849 confirmed, 3629 partial, 0 refuted)
  [Orchestrator] Verification significance ratio: 33.7% (threshold: 50.0%)

 ======================================================================
  Step 6: LEARN — Developer Copilot Agent
 ======================================================================
  [Orchestrator] Copilot: 4,534 points to 7 teams in 5.1s
  [Orchestrator] Leaderboard:
  [Orchestrator]   #1 eu-backend: 1349 pts
  [Orchestrator]   #2 eu-frontend: 1171 pts
  [Orchestrator]   #3 platform: 588 pts
  [Orchestrator]   #4 backend-api: 438 pts
```

copilot/generate-llm-rationales ⊗ 0 ⚠ 0 sarthakc123 (2 weeks ago) Ln 475, Col 3 Spaces: 2 UTF-8 LF {} Markdown

Refined System Concept

Simulation Days

30

Total Jobs

24,274

Carbon Price

$75/ton CO_2e

Navigation

- Your Impact This Week
- Overview
- Carbon Analysis
- Optimization Results
- Verification (MRV)
- Team Leaderboard
- Evidence Explorer
- Trade-off Analysis
- Agent Reasoning
- The Debate
- The Impact
- Ask the Agent

Deploy

System Overview

Closed-loop carbon optimization: Sense → Model → Decide → Act → Verify → Learn

Baseline Emissions	Optimized Emissions	Reduction	Verified Savings
373.8 $kgCO_2e$	311.8 $kgCO_2e$ ↓ -62.0 $kgCO_2e$	16.6%	62045 gCO_2e

Recommendations	Approved	Executed	Verified	Points Awarded
5478	5478	5478	5478	4,534

Cost Comparison

Baseline Cloud Cost	Optimized Cloud Cost	Baseline Effective Cost
$8,591.19	$8,544.95 ↑ $-46.23	$8,619.22

Simulation Days

30

Total Jobs

24,274

Carbon Price

$75/ton CO_2e

Navigation

- Your Impact This Week
- Overview
- Carbon Analysis
- Optimization Results
- Verification (MRV)
- Team Leaderboard
- Evidence Explorer
- Trade-off Analysis
- Agent Reasoning
- The Debate
- The Impact
- Ask the Agent

Deploy

Ask the Agent

Chat with the Carbon Optimization Assistant — ask about your pipeline results, the methodology, or carbon reduction strategies.

Live mode — using groq for responses.

Clear

What drove the most carbon savings?

The optimization reduced your cloud emissions by shifting workloads to cleaner regions and time windows. The biggest lever is region-shifting: moving batch jobs from high-intensity regions (e.g. us-east-1 at ~415 gCO_2/kWh) to low-intensity ones (e.g. eu-north-1 at ~8 gCO_2/kWh) can cut per-job emissions by over 90%. Time-shifting within the same region is lower-impact but zero-cost. See the **Carbon Analysis** page for a full breakdown by region, team, and workload type.

Ask about your carbon footprint, agents, or methodology…

Risks & Next Steps

Risks Identified

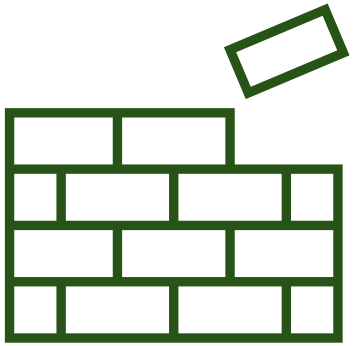
- Unreliable emissions data
- LLM hallucinations
- Orchestration instability

Safeguards

- Deterministic carbon calculations
- Verified emissions data sources
- Termination safeguards for agent workflows

Checkpoint 3 Goal:

Working prototype that integrates real though based decision making and real emissions data and improves the stability of the agent system.



THANK YOU